

시작하기 전에

1. Git 레포지토리

이 책에서는 두 개의 Git 레포지토리를 사용합니다.

레포	용도	주소
rag-start	실습용. import와 데이터가 준비된 파일에 TODO를 채워넣습니다	https://github.com/metacoding-18-ai-applied-v4/rag-start
rag-end	완성 코드. 막히면 여기서 정답을 확인합니다	https://github.com/metacoding-18-ai-applied-v4/rag-end

1.1 실습 흐름

1. **rag-start** 레포를 클론합니다.
2. 챗터를 보면서 **[실습]** 파일의 TODO 부분에 코드를 작성합니다.
3. 막히면 **rag-end** 완성 코드를 참고합니다.

```
git clone https://github.com/metacoding-18-ai-applied-v4/rag-start.git
cd rag-start
```

1.2 폴더 구조

각 챗터는 하나의 예제 폴더에 대응합니다. 챗터를 진행할수록 폴더 번호가 올라가며 시스템이 한 단계씩 성장합니다.

rag-start/		
├── ex01/	←	CH01: 환각과 RAG의 첫 만남
├── ex02/	←	CH02: 일단 사내 시스템부터
├── ex04/	←	CH04: 문서를 지식으로 바꾸다
├── ex05/	←	CH05: 드디어 답해준다
├── ex06/	←	CH06: 연차도 규정도 한번에
├── ex07/	←	CH07: 실제로 써보니
├── ex08/	←	CH08: 엉뚱한 문서를 가져온다
├── ex09/	←	CH09: 질문을 제대로 이해 못한다

ex10/ ← CH10: PDF 이미지까지 잡아라
ex11/ ← CH11: 커넥트HR 에이전트의 완성

CH03은 이야기 챕터로 코드가 없습니다.

1.3 코드 분류

각 챕터의 코드 파일에는 세 가지 분류가 붙어 있습니다.

분류	의미	rag-start 상태	독자 액션
[실습]	챕터 핵심 코드	import + 데이터 + TODO 주석	TODO를 채워 코드 작성
[설명]	중요하지만 핵심은 아닌 코드	완성 코드 그대로	코드를 읽고 이해
[참고]	이 챕터 주제가 아닌 코드	완성 코드 그대로	파일명과 한 줄 설명만 확인

[실습] 파일에는 import, 상수, 더미 데이터, 테스트 입력값이 미리 준비되어 있습니다. 챕터를 따라 하며 #
TODO: 주석 부분만 채워넣으면 됩니다. [설명] 과 [참고] 파일은 실행에 필요한 완성 코드가 이미 들어
있으므로 수정하지 않습니다.

2. 환경 설정

2.1 Python 설치

이 책의 모든 예제는 Python 3.12 를 기준으로 작성되었습니다. 3.10~3.12에서 동작하며 3.13 이상에서는 일부
패키지 호환성 문제가 있을 수 있습니다.

macOS

```
# Homebrew로 설치
brew install python@3.12
```

Windows

공식 사이트(<https://www.python.org/downloads/>)에서 Python 3.12를 다운로드합니다. 설치 시 "Add
Python to PATH" 체크박스를 반드시 선택하세요.

2.2 가상환경 설정

예제마다 패키지 버전이 다를 수 있으므로 반드시 가상환경을 만들어서 진행하세요.

```
# 가상환경 생성
python3.12 -m venv .venv

# 활성화 (macOS/Linux)
source .venv/bin/activate

# 활성화 (Windows)
.venv\Scripts\activate

# 패키지 설치
pip install -r requirements.txt
```

가상환경이 활성화되면 터미널 프롬프트 앞에 `(.venv)` 가 표시됩니다.

2.3 Ollama 설치

Ollama는 로컬 LLM을 실행하는 도구입니다. 공식 사이트(<https://ollama.com>)에서 설치하세요.

```
# 설치 확인
ollama --version
```

2.4 LLM 모델 다운로드

챕터별로 사용하는 모델이 다릅니다.

모델	사용 챕터	용도	RAM
<code>deepseek-r1:8b</code>	CH01~05	추론(Chain-of-Thought) 특화	16GB 이상
<code>nomic-embed-text</code>	CH01	임베딩 (CH04부터 ko-sroberta로 교체)	—
<code>llama3.1:8b</code>	CH06~CH11	Tool Calling 지원	16GB 이상
<code>qwen2.5vl:7b</code>	CH10·CH11	비전 LLM (스캔 PDF 처리)	16GB 이상

```
# CH01~05 모델
ollama pull deepseek-r1:8b
ollama pull nomic-embed-text

# CH06~CH11 모델
ollama pull llama3.1:8b

# CH10·CH11 비전 모델
ollama pull qwen2.5vl:7b
```

RAM이 부족하거나 응답이 너무 느리면: `.env` 에서 `LLM_PROVIDER=openai` 로 바꿔서 GPT-4o-mini를 쓸 수 있습니다. 단, API 비용이 발생합니다.

2.5 .env 파일 설정

각 예제 폴더의 `.env.example` 을 `.env` 로 복사한 뒤 값을 채워넣으세요.

```
cp .env.example .env
```

```
# LLM 설정
LLM_PROVIDER=ollama
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=deepseek-r1:8b          # CH01~05
# OLLAMA_MODEL=llama3.1:8b          # CH06~CH11 (Tool Calling 필요)

# OpenAI 사용 시
# LLM_PROVIDER=openai
# OPENAI_API_KEY=sk- ...
# OPENAI_MODEL=gpt-4o-mini

# PostgreSQL (ex02 이후)
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=rag_db
POSTGRES_USER=rag_user
POSTGRES_PASSWORD=rag_password

# 벡터DB + 임베딩
CHROMA_PERSIST_DIR=./data/chroma_db
EMBEDDING_MODEL=jhgan/ko-sroberta-multitask
```

```
# 비전 LLM (ex10·ex11)
VISION_MODEL=qwen2.5vl:7b
VISION_PROVIDER=ollama
# VISION_PROVIDER=openai # 로컬 사양 부족 시
```

2.6 Docker (PostgreSQL)

ex02 이후 예제는 PostgreSQL이 필요합니다. Docker Compose로 실행합니다.

```
docker compose up -d
```

PostgreSQL 16 Alpine 이미지를 사용하며 `data/schema.sql` 이 자동으로 초기화됩니다.

2.7 핵심 패키지 요약

패키지	버전	용도
<code>langchain</code>	0.3.x	RAG 파이프라인, 에이전트
<code>chromadb</code>	1.5.x	벡터 데이터베이스
<code>fastapi</code>	0.115.x	API 서버
<code>sentence-transformers</code>	3.3.x	한국어 임베딩 + Cross-Encoder Reranker (CH11)
<code>psycopg2-binary</code>	2.9.x	PostgreSQL 연결
<code>pypdf</code>	4.3.x	PDF 파싱
<code>python-docx</code>	1.1.x	DOCX 파싱
<code>openpyxl</code>	3.1.x	XLSX 파싱
<code>rank-bm25</code>	0.2.x	하이브리드 검색 (CH08·CH11)
<code>easyocr</code>	1.7.x	OCR (CH10)

각 예제 폴더의 `requirements.txt` 로 한 번에 설치할 수 있습니다.

```
pip install -r requirements.txt
```

3. 자주 만나는 오류

3.1 `python` 명령어가 안 될 때

macOS/Linux에서는 `python` 대신 `python3.12` 를 사용해야 할 수 있습니다.

```
# python이 안 되면
python3.12 --version
python3.12 -m venv .venv
```

3.2 `pip install` 에서 권한 오류

가상환경 없이 시스템 Python에 설치하려고 하면 권한 오류가 발생합니다. 가상환경을 먼저 활성화하세요.

```
# 이렇게 하면 안 됩니다
pip install langchain # PermissionError 또는 externally-managed-environment

# 이렇게 하세요
source .venv/bin/activate # 먼저 가상환경 활성화
pip install -r requirements.txt
```

3.3 `pip` 대신 `pip3`

`pip` 명령이 안 되면 `pip3` 를 사용하세요. 가상환경 안에서는 둘 다 동일합니다.

3.4 `psycopg2-binary` 설치 실패 (macOS Apple Silicon)

M1/M2/M3 Mac에서 `psycopg2-binary` 설치가 실패할 수 있습니다.

```
# libpq 먼저 설치
brew install libpq
pip install psycopg2-binary
```

3.5 Ollama 연결 오류

```
ConnectionRefusedError: Connection refused
```

Ollama 서버가 실행 중인지 확인하세요.

```
# Ollama 서버 실행  
ollama serve
```

```
# 또는 Ollama 앱을 실행하면 자동으로 서버가 시작됩니다
```

3.6 모델 다운로드 오류

```
model not found
```

해당 모델을 아직 다운로드하지 않은 경우입니다. `ollama pull 모델명` 으로 다운로드하세요.